

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

ЛАБОРАТОРНАЯ РАБОТА № 8

По курсу Проектирование пользовательских интерфейсов в веб
Продвинутый GIT

ТЕМА
**САЙТ ДЛЯ ГРУППЫ КОМПАНИЙ ОПТОВОЙ ПРОДАЖИ РЫБНОЙ
ПРОДУКЦИИ**

Выполнил: Балынин Е.Д. 241-3210
Проверил(а): Даньшина М.В.

Цель работы

Изучить продвинутые возможности системы контроля версий Git: работу с историей коммитов, ветками, сташем, интерактивным rebase и инструментами отладки.

Задачи

1. Изучить устройство рабочего каталога, индекса, репозитория, роль HEAD и веток.
2. Ознакомиться с файлом .gitignore и командой `git rm --cached`.
3. Изучить навигацию по истории: `git checkout`, `git switch`, `git log`.
4. Изучить отмену и изменение коммитов: `git revert`, `git reset`, `git commit --amend`.
5. Изучить команды `git diff`, `git blame`, `git stash`, `git cherry-pick`.
6. Изучить продвинутые операции: `git rebase`, `git bisect`, `git reflog`.

Отчет по выполнению

1. Устройство рабочего каталога, индекса, репозитория, роль HEAD и веток

Git работает с тремя зонами: **Working directory** (файлы на диске), **Index / Staging area** (изменения подготовленные к коммиту через `git add`) и **Repository** (база данных объектов в `.git/`). **HEAD** — указатель на текущий коммит, **ветка** — именованный указатель на конкретный коммит, который смещается при каждом новом коммите.

```
git status      # состояние рабочего каталога и индекса
ls .git/        # содержимое репозитория
cat .git/HEAD   # на что указывает HEAD
```

A screenshot of a terminal window with a dark background. The terminal shows the output of two commands: 'git status' and 'ls .git'. The 'git status' output indicates that there are untracked files in the 'docs/' directory: 'report_git_lab.md' and '~\$port_git_lab.md'. It also states that nothing was added to the commit. The 'ls .git' output shows the structure of the .git directory, including subdirectories like 'COMMIT_EDITMSG', 'HEAD', 'FETCH_HEAD', 'config', 'description', 'hooks', 'index', 'info', 'logs', 'objects', 'packed-refs', and 'refs'. The terminal window has a title bar with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', 'PORTS', 'GITLENS', and 'PLAYWRIGHT'. The 'TERMINAL' tab is active. The terminal prompt is 'x1lefg0r@x1lefg0r-OSX edutrack %'.

Рисунок 1 – Вывод `git status` и `ls .git/`

2. Файл .gitignore

Файл `.gitignore` содержит список файлов и директорий, которые Git **не отслеживает**. Поддерживает glob-паттерны: `*.log`, `dist/`, `**/*.env`. Применяется для исключения `node_modules`, файлов сборки и конфиденциальных данных.

```
cat .gitignore # содержимое файла игнорирования
git status     # убедиться, что node_modules не отслеживается
```

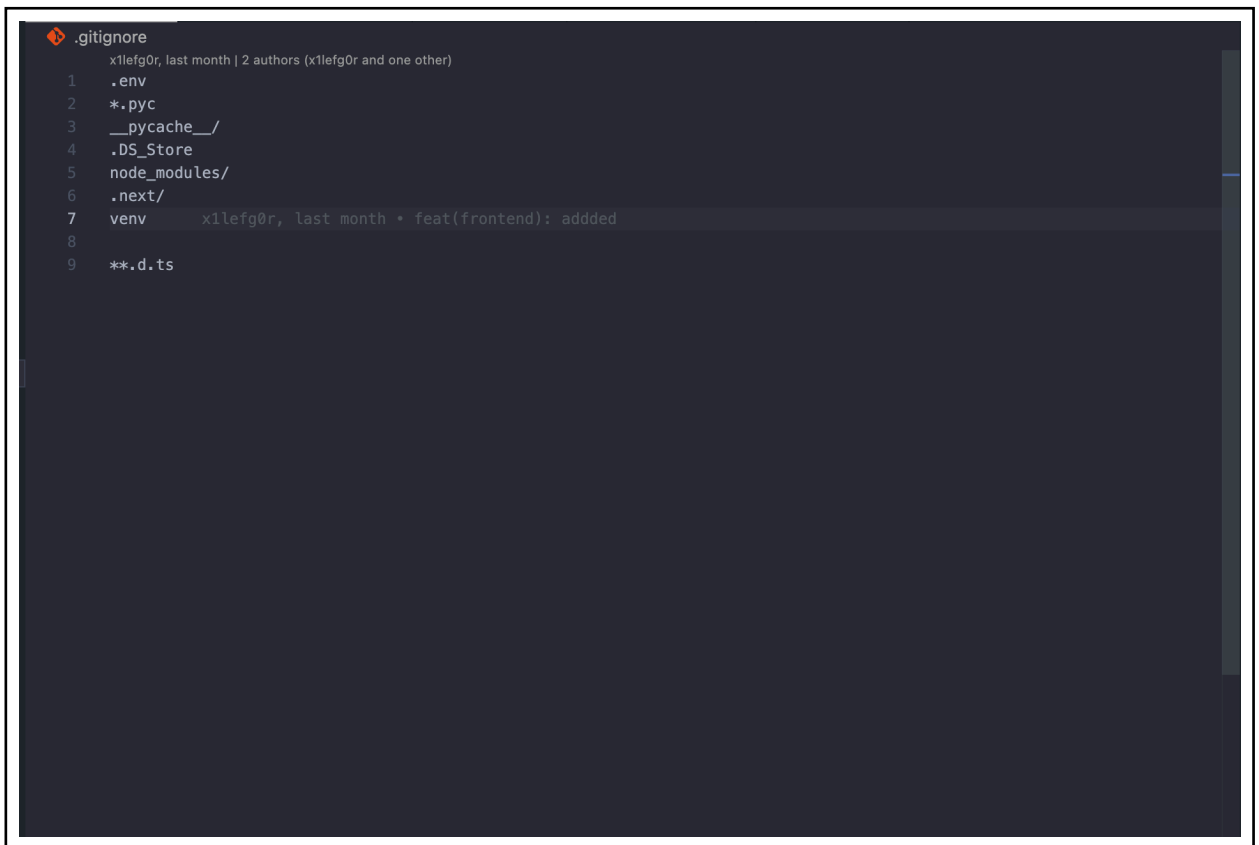


Рисунок 2 – Содержимое .gitignore и вывод git status

3. Переход к конкретному коммиту — git checkout <hash>

Перемещает HEAD на указанный коммит. HEAD переходит в состояние **detached HEAD** — не привязан к ветке. Изменения не теряются, это режим просмотра. Для возврата используется git switch <ветка>.

```
git log --oneline          # смотрим историю, выбираем хэш  
git checkout af7f0fa      # переходим к коммиту (detached HEAD)  
cat .git/HEAD             # содержит хэш, а не ref  
git switch git-lab-demo   # возвращаемся на ветку
```

```

x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline
c058d5c (HEAD -> git-lab-demo, origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack % git checkout 1ea8432
A       screenshots/1.png
A       screenshots/2.png
Note: switching to '1ea8432'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 1ea8432 fix: bugs
x1lefg0r@x1lefg0r-OSX edutrack % cat .git/HEAD
1ea843207c910545c5aafcd6223199e70152a5cc
x1lefg0r@x1lefg0r-OSX edutrack % git branch
* (HEAD detached at 1ea8432)
  git-lab-demo
  master
x1lefg0r@x1lefg0r-OSX edutrack % git checkout git-lab-demo
A       screenshots/1.png
A       screenshots/2.png
Previous HEAD position was 1ea8432 fix: bugs
Switched to branch 'git-lab-demo'
x1lefg0r@x1lefg0r-OSX edutrack %

```

Рисунок 3 – Предупреждение о detached HEAD при git checkout <hash>

4. Переход на другую ветку — git checkout <branch>, git switch <branch>
 git switch — современная замена git checkout для переключения веток. Флаг -с создаёт новую ветку и сразу переключается на неё. git checkout -b делает то же самое в старом синтаксисе.

```

git branch                                # список веток
git checkout -b demo-branch-1             # создать и переключиться (старый способ)
git switch git-lab-demo                   # вернуться (новый способ)
git switch -c demo-branch-2               # создать и переключиться (новый способ)
git switch git-lab-demo
git branch -d demo-branch-1 demo-branch-2 # удалить тестовые ветки

```

```
x1lefg0r@x1lefg0r-OSX edutrack % git branch
* (HEAD detached at 1ea8432)
  git-lab-demo
  master
x1lefg0r@x1lefg0r-OSX edutrack % git checkout git-lab-demo
A   screenshots/1.png
A   screenshots/2.png
Previous HEAD position was 1ea8432 fix: bugs
Switched to branch 'git-lab-demo'
x1lefg0r@x1lefg0r-OSX edutrack % git branch
* git-lab-demo
  master
x1lefg0r@x1lefg0r-OSX edutrack % git checkout -b example
Switched to a new branch 'example'
x1lefg0r@x1lefg0r-OSX edutrack % git switch master
A   screenshots/1.png
A   screenshots/2.png
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
x1lefg0r@x1lefg0r-OSX edutrack % git switch -c example-switch
Switched to a new branch 'example-switch'
x1lefg0r@x1lefg0r-OSX edutrack % git switch master
A   screenshots/1.png
A   screenshots/2.png
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
x1lefg0r@x1lefg0r-OSX edutrack % git branch -d example example-switch
Deleted branch example (was c058d5c).
Deleted branch example-switch (was c058d5c).
x1lefg0r@x1lefg0r-OSX edutrack % git branch
  git-lab-demo
* master
x1lefg0r@x1lefg0r-OSX edutrack % git checkout git-lab-demo
A   screenshots/1.png
A   screenshots/2.png
Switched to branch 'git-lab-demo'
x1lefg0r@x1lefg0r-OSX edutrack % |
```

Рисунок 4 – Создание веток и переключение между ними

5. Отмена изменений через новый коммит — `git revert <hash>`

`git revert` создаёт **новый коммит**, отменяющий изменения указанного. История не переписывается — безопасный способ отмены для коммитов, уже отправленных в удалённый репозиторий.

```
git log --oneline                # выбираем коммит для отмены
git revert 2d9b048 --no-edit      # отменяем "fix: fixed pagination
page"
git log --oneline                # новый revert-коммит в истории
git revert HEAD --no-edit        # отменяем revert (возвращаем как
было)
```

```

fatal: unrecognized argument: --online
x11efg0r@x11efg0r-OSX edutrack % git log --online
ecb84a2 (HEAD -> git-lab-demo) chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some a11y
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x11efg0r@x11efg0r-OSX edutrack % git revert c058d5c
[git-lab-demo 60c297f] Revert "chore: added readme"
1 file changed, 111 deletions(-)
delete mode 100644 README.md
x11efg0r@x11efg0r-OSX edutrack % git log --online
60c297f (HEAD -> git-lab-demo) Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some a11y
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x11efg0r@x11efg0r-OSX edutrack % git revert HEAD --no-edit
[git-lab-demo 6b4ea9a] Reapply "chore: added readme"
Date: Sun May 31 22:41:48 2026 +0300
1 file changed, 111 insertions(+)
create mode 100644 README.md
x11efg0r@x11efg0r-OSX edutrack % git log --online
6b4ea9a (HEAD -> git-lab-demo) Reapply "chore: added readme"
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some a11y
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x11efg0r@x11efg0r-OSX edutrack %

```

Рисунок 5 – `git log --online` с появившимся revert-коммитом

6. Режим `--soft` и `--hard` для `git reset`

`git reset` перемещает HEAD и ветку назад по истории. Режим `--soft` отменяет коммит, но оставляет изменения в индексе. Режим `--hard` полностью удаляет изменения из рабочего каталога — необратимая операция без `git reflog`.

`--soft`: изменения остаются в staging

```
git reset --soft HEAD~1
```

```
git status
```

файлы в staged

```
git commit -m "chore: added readme"
```

`--hard`: изменения удаляются полностью

```
git reset --hard HEAD~1
```

```
git log --online
```

коммита нет

```
git reset --hard c058d5c
```

восстановить по хэшу из reflog

```

2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack % git reset --soft HEAD~1
x1lefg0r@x1lefg0r-OSX edutrack % git status
On branch git-lab-demo
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    screenshots/5.png

x1lefg0r@x1lefg0r-OSX edutrack % git commit -m "chore: added readme"
[git-lab-demo e25d71a] chore: added readme
 1 file changed, 111 insertions(+)
 create mode 100644 README.md
x1lefg0r@x1lefg0r-OSX edutrack % git status
On branch git-lab-demo
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    example
    screenshots/5.png

nothing added to commit but untracked files present (use "git add" to track)
x1lefg0r@x1lefg0r-OSX edutrack % git add example
x1lefg0r@x1lefg0r-OSX edutrack % git commit -m "example"
[git-lab-demo 25311d0] example
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 example
x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline
25311d0 (HEAD -> git-lab-demo) example
e25d71a chore: added readme
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack % git reset --hard HEAD~1
HEAD is now at e25d71a chore: added readme
x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline
e25d71a (HEAD -> git-lab-demo) chore: added readme
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack %

```

Рисунок 6 – git status после --soft (файлы в staged) и после --hard (чисто)

7. Изменить сообщение последнего коммита — git commit --amend

--amend перезаписывает последний коммит: изменяет сообщение и/или добавляет файлы из индекса. Создаёт новый объект коммита с новым хэшем. **Не применять для коммитов, уже отправленных на удалённый репозиторий.**

```
git log --oneline
```

```
git commit --amend -m "chore: added readme" # новое сообщение без редактора
```

```
git log --oneline # хэш изменился
```



```

x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline
e25d71a (HEAD -> git-lab-demo) chore: added readme
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack % git commit --amend -m "chore: added readme lab"
[git-lab-demo cdf3046] chore: added readme lab
Date: Sun May 31 22:43:31 2026 +0300
1 file changed, 111 insertions(+)
create mode 100644 README.md
x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline
cdf3046 (HEAD -> git-lab-demo) chore: added readme lab
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack % |

```

Рисунок 7 – git log --oneline до и после --amend

8. Содержимое HEAD — cat .git/HEAD

.git/HEAD — текстовый файл, всегда указывающий на текущую позицию. В нормальном режиме содержит ref: refs/heads/<ветка>. В режиме detached HEAD содержит SHA-1 хэш коммита напрямую.

```

cat .git/HEAD                # ref: refs/heads/git-lab-demo
git checkout af7f0fa
cat .git/HEAD                # af7f0fa... (хэш — detached HEAD)
git switch git-lab-demo
cat .git/HEAD                # снова ref: refs/heads/git-lab-demo

```

```

x1lefg0r@x1lefg0r-OSX edutrack % cat .git/HEAD
ref: refs/heads/git-lab-demo
x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline
cdf3046 (HEAD -> git-lab-demo) chore: added readme lab
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack % git checkout 6f111a4
Note: switching to '6f111a4'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 6f111a4 feat: added openapi schema
x1lefg0r@x1lefg0r-OSX edutrack % cat .git/HEAD
6f111a48e09f27b41fd5444af5f60dc1e9ea06d6
x1lefg0r@x1lefg0r-OSX edutrack % git switch master
Previous HEAD position was 6f111a4 feat: added openapi schema
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
x1lefg0r@x1lefg0r-OSX edutrack % git branch
  git-lab-demo
* master
x1lefg0r@x1lefg0r-OSX edutrack % git switch git-lab-demo
Switched to branch 'git-lab-demo'
x1lefg0r@x1lefg0r-OSX edutrack %

```

Рисунок 8 – cat .git/HEAD в обычном режиме и в detached HEAD

9. Куда указывает ветка — cat .git/refs/heads/master

Каждая ветка — это файл в .git/refs/heads/, содержащий SHA-1 хэш коммита, на который ветка указывает. При создании нового коммита файл ветки обновляется автоматически.

```

cat .git/refs/heads/master # хэш последнего коммита ветки
git log --oneline -1       # должен совпадать

```

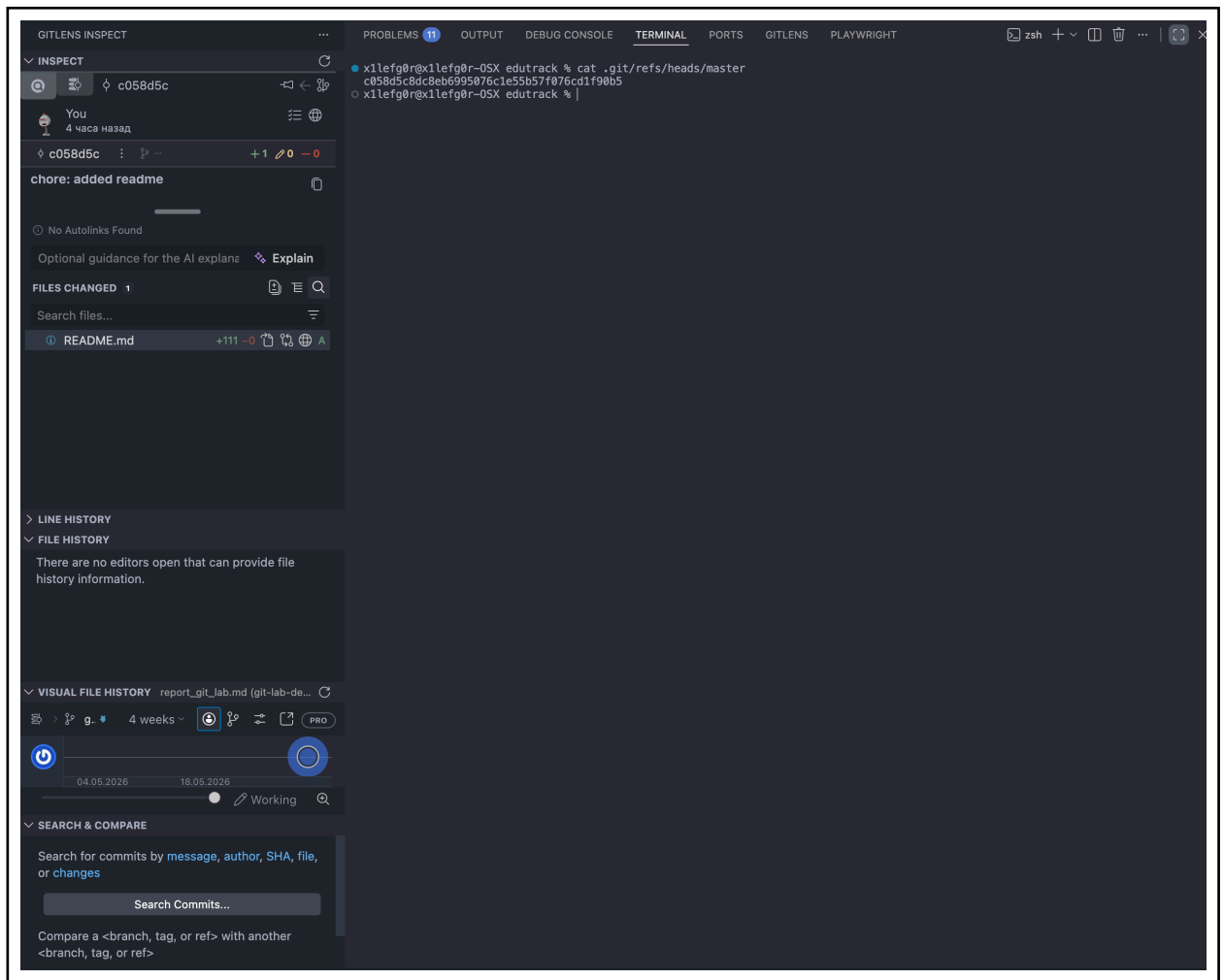


Рисунок 9 – Совпадение хэша из файла ветки и из git log

10. Команда git log --oneline

Компактный вывод истории: каждый коммит в одну строку (короткий хэш + сообщение). Флаг --graph добавляет ASCII-граф веток, --all показывает все ветки, -N ограничивает количество коммитов.

```
git log --oneline           # вся история
git log --oneline -5       # последние 5 коммитов
git log --oneline --graph --all # граф всех веток
```

```

x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline
cdf3046 (HEAD -> git-lab-demo) chore: added readme lab
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline -5
cdf3046 (HEAD -> git-lab-demo) chore: added readme lab
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c (origin/master, origin/HEAD, master) chore: added readme
6f111a4 feat: added openapi schema
x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline --graph --all
* cdf3046 (HEAD -> git-lab-demo) chore: added readme lab
* 60c297f Revert "chore: added readme"
* ecb84a2 chore: docs
* c058d5c (origin/master, origin/HEAD, master) chore: added readme
* 6f111a4 feat: added openapi schema
* af7f0fa refactor: added some ally
* 2d9b048 fix: fixed pagination page
* 1ea8432 fix: bugs
* f4e2c0a fix: small bug with achievements
* 2e6724c feat: added ollama to seeder and fixes small bugs
* ee6e0ba feat(frontend): added
* 360b491 chore: install shadcn
* e5b282a chore: added shadcn
* 540d787 chore: init
* b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack %

```

Рисунок 10 – Вывод git log --oneline --graph --all

11. Команды git diff и git diff <hash>

git diff показывает построчную разницу между состояниями. Без аргументов — изменения не в индексе. --staged — изменения в индексе. С хэшем — разница между текущим состоянием и указанным коммитом.

git diff	# изменения не добавленные в staging
git diff --staged	# изменения в staging
git diff HEAD~1 HEAD	# разница между последними двумя
КОММИТАМИ	
git diff 1ea8432 c058d5c	# разница между двумя конкретными
КОММИТАМИ	



Рисунок 11 – Вывод `git diff HEAD~1 HEAD`

12. Команда `git rm --cached <файл>`

Удаляет файл из индекса (перестает отслеживаться Git), но **оставляет его на диске**. Используется когда файл уже закоммичен, а его нужно добавить в `.gitignore`.

```
touch test_temp.txt
git add test_temp.txt
git status                # файл в staged
git rm --cached test_temp.txt # убираем из индекса
git status                # файл теперь untracked
rm test_temp.txt
```

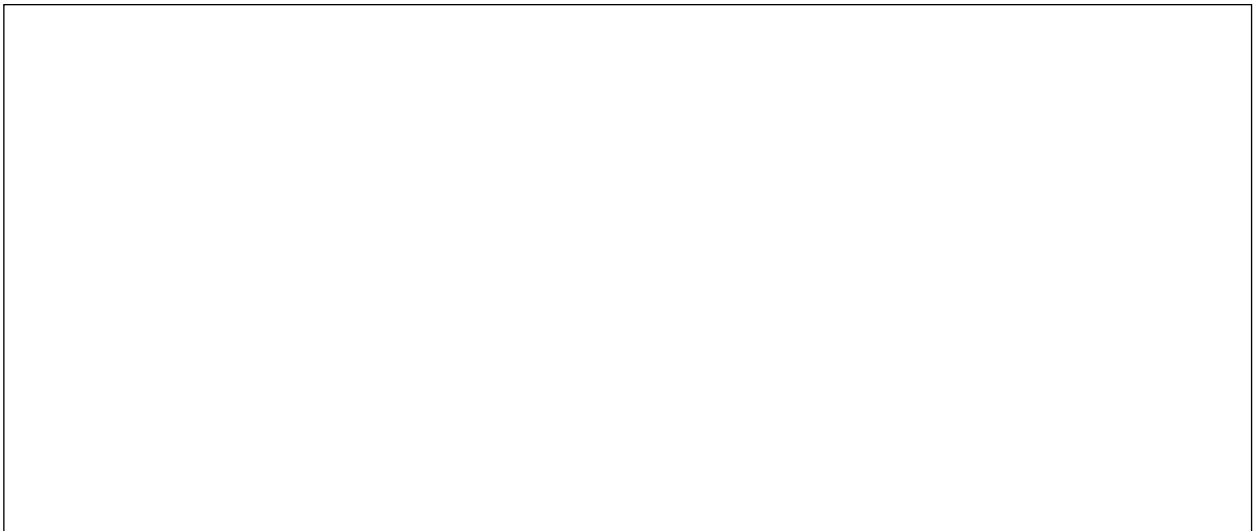
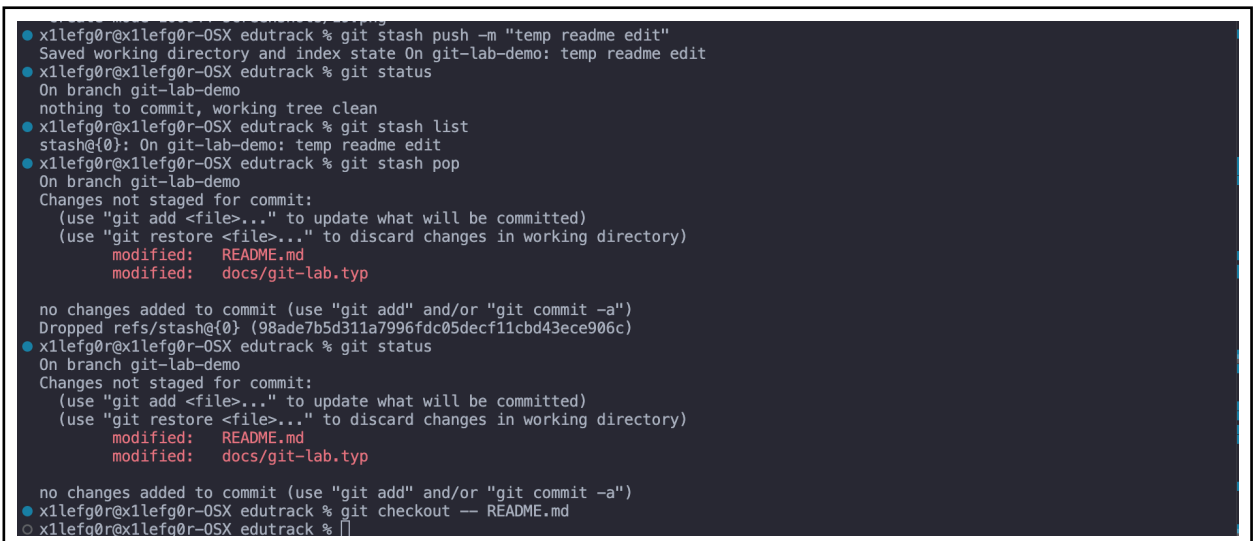


Рисунок 12 – `git status` до и после `git rm --cached`

13. Команда git stash

Временно сохраняет незакоммиченные изменения в стек, очищая рабочий каталог. Позволяет переключиться на другую задачу без коммита. pop восстанавливает и удаляет из стека, apply — восстанавливает без удаления.

```
echo "test" >> README.md
git stash push -m "temp readme edit" # спрятать с названием
git status                          # чисто
git stash list                       # список спрятанного
git stash pop                        # восстановить последнее
git stash apply stash@{0}            # применить без удаления из
стека
git checkout -- README.md           # откат тестового изменения
```



```
x1lefg0r@x1lefg0r-OSX edutrack % git stash push -m "temp readme edit"
Saved working directory and index state On git-lab-demo: temp readme edit
x1lefg0r@x1lefg0r-OSX edutrack % git status
On branch git-lab-demo
nothing to commit, working tree clean
x1lefg0r@x1lefg0r-OSX edutrack % git stash list
stash@{0}: On git-lab-demo: temp readme edit
x1lefg0r@x1lefg0r-OSX edutrack % git stash pop
On branch git-lab-demo
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
        modified:   docs/git-lab.typ
no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (98ade7b5d311a7996fdc05decf11cbd43ece906c)
x1lefg0r@x1lefg0r-OSX edutrack % git status
On branch git-lab-demo
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
        modified:   docs/git-lab.typ
no changes added to commit (use "git add" and/or "git commit -a")
x1lefg0r@x1lefg0r-OSX edutrack % git checkout -- README.md
x1lefg0r@x1lefg0r-OSX edutrack %
```

Рисунок 13 – git stash list и git stash pop

14. Восстановление удалённого через reset --hard коммита

git reflog хранит лог всех перемещений HEAD, включая «потерянные» после reset --hard коммиты. Позволяет восстановить их в течение 90 дней по хэшу из рефлога.

```
git reset --hard HEAD~1            # "удаляем" последний коммит
git log --oneline                  # коммита нет
git reflog                         # находим хэш в reflog
git reset --hard c058d5c           # восстанавливаем
git log --oneline                  # коммит вернулся
# Альтернатива: git switch -c recovered c058d5c
```

```

c058d5c chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
● x1lef0r@x1lef0r-OSX edutrack % git reset --hard HEAD~1
HEAD is now at 579053e test
● x1lef0r@x1lef0r-OSX edutrack % git log --oneline
579053e (HEAD -> git-lab-demo) test
cdf3046 chore: added readme lab
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
● x1lef0r@x1lef0r-OSX edutrack % git relog
579053e (HEAD -> git-lab-demo) HEAD@{0}: reset: moving to HEAD~1
5bc7cc7 HEAD@{1}: reset: moving to HEAD
5bc7cc7 HEAD@{2}: commit: chore: docs
579053e (HEAD -> git-lab-demo) HEAD@{3}: checkout: moving from 2d9b048d71bef6e3d07b8413e64b05918f28b86 to git-lab-demo
2d9b048 HEAD@{4}: checkout: moving from git-lab-demo to 2d9b048d71bef6e3d07b8413e64b05918f28b86
579053e (HEAD -> git-lab-demo) HEAD@{5}: checkout: moving from git-lab-demo to git-lab-demo
579053e (HEAD -> git-lab-demo) HEAD@{6}: checkout: moving from feature-a to git-lab-demo
efccbc0 (master) HEAD@{7}: rebase (finish): returning to refs/heads/feature-a
efccbc0 (master) HEAD@{8}: rebase (start): checkout master
efccbc0 (master) HEAD@{9}: checkout: moving from master to feature-b
efccbc0 (master) HEAD@{10}: checkout: moving from feature-a to master
1ea8432 HEAD@{11}: checkout: moving from master to feature-a
efccbc0 (master) HEAD@{12}: commit: chore: docs
cdf3046 HEAD@{13}: reset: moving to HEAD~1
34b2f8c HEAD@{14}: commit: chore: squashed last 2 commits
cdf3046 HEAD@{15}: reset: moving to HEAD~2
58e9f29 (origin/master, origin/HEAD) HEAD@{16}: reset: moving to 58e9f29027cac44115ed953bae0500122c4aa3e6
● x1lef0r@x1lef0r-OSX edutrack % git reset --hard 5bc7cc7
HEAD is now at 5bc7cc7 chore: docs
● x1lef0r@x1lef0r-OSX edutrack % git log --oneline
5bc7cc7 (HEAD -> git-lab-demo) chore: docs
579053e test
cdf3046 chore: added readme lab
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
○ x1lef0r@x1lef0r-OSX edutrack % |

```

Рисунок 14 – git relog с видимым «потерянным» коммитом

15. Команда git cherry-pick

Применяет изменения из конкретного коммита другой ветки в текущую. Создаёт новый коммит с теми же изменениями, но новым хэшем. В отличие от merge, берётся ровно один выбранный коммит.

```

git cherry-pick af7f0fa      # перенести коммит "refactor: added
some ally"
git log --oneline           # перенесённый коммит в истории
git revert HEAD --no-edit   # отменяем cherry-pick

```

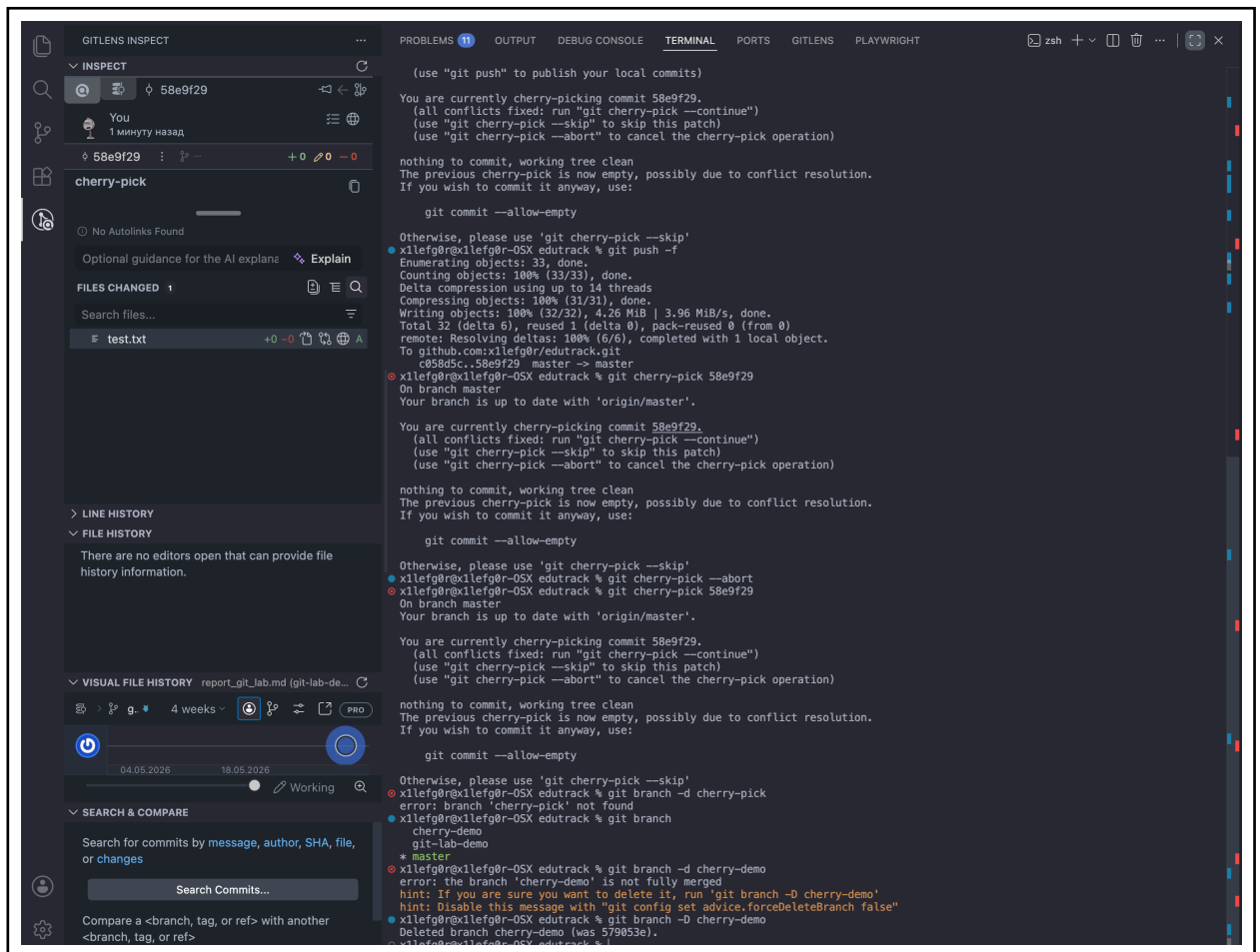


Рисунок 15 – git log --oneline с перенесённым коммитом

16. Объединение нескольких коммитов в один

git reset --soft HEAD~N разворачивает N коммитов, оставляя изменения в staging — затем создаётся один новый коммит. git rebase -i HEAD~N позволяет интерактивно выбрать коммиты: squash объединяет с сохранением сообщений, fixup — без.

```
# Через reset --soft
git log --oneline
git reset --soft HEAD~2 # разворачиваем 2 коммита
git status # изменения в staged
git commit -m "squashed commit message"

# Через rebase -i (откроется редактор)
git rebase -i HEAD~3 # меняем pick → squash для нужных коммитов
```



```

x1lefg0r@x1lefg0r-OSX edutrack % git status
On branch master
Your branch is behind 'origin/master' by 2 commits, and can be fast-forwarded.
(use "git pull" to update your local branch)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   screenshots/10.png
    new file:   screenshots/11.png
    new file:   screenshots/12.png
    new file:   screenshots/13.png
    new file:   screenshots/14.png
    new file:   screenshots/5.png
    new file:   screenshots/6.png
    new file:   screenshots/7.png
    new file:   screenshots/8.png
    new file:   screenshots/9.png
    new file:   test.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    screenshots/15.png

x1lefg0r@x1lefg0r-OSX edutrack % git commit -m "chore: squashed last 2 commits"
[master 34b2f8c] chore: squashed last 2 commits
11 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 screenshots/10.png
create mode 100644 screenshots/11.png
create mode 100644 screenshots/12.png
create mode 100644 screenshots/13.png
create mode 100644 screenshots/14.png
create mode 100644 screenshots/5.png
create mode 100644 screenshots/6.png
create mode 100644 screenshots/7.png
create mode 100644 screenshots/8.png
create mode 100644 screenshots/9.png
create mode 100644 test.txt

x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline
34b2f8c (HEAD -> master) chore: squashed last 2 commits
cdf3046 chore: added readme lab
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init

x1lefg0r@x1lefg0r-OSX edutrack % git reset --soft HEAD~1
x1lefg0r@x1lefg0r-OSX edutrack % git commit -m "chore: docs"
[master efcabc0] chore: docs
11 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 screenshots/10.png
create mode 100644 screenshots/11.png
create mode 100644 screenshots/12.png
create mode 100644 screenshots/13.png
create mode 100644 screenshots/14.png
create mode 100644 screenshots/5.png
create mode 100644 screenshots/6.png
create mode 100644 screenshots/7.png
create mode 100644 screenshots/8.png
create mode 100644 screenshots/9.png
create mode 100644 test.txt
x1lefg0r@x1lefg0r-OSX edutrack % |

```

Рисунок 16 – git log --oneline до и после squash

17. Перенос коммитов — git rebase --onto

git rebase --onto <newbase> <upstream> <branch> переносит серию коммитов из upstream..branch и применяет их поверх newbase. Полезно, если ветка была ответвлена не от того места.

```

git checkout -b feature-demo 1ea8432 # ветка от старого коммита
git switch git-lab-demo
git rebase --onto git-lab-demo 1ea8432 feature-demo
git log --oneline feature-demo # коммиты теперь поверх git-
lab-demo
git branch -d feature-demo

```

```

x1lefg0r@x1lefg0r-OSX edutrack % git checkout -b feature-a 1ea8432
Switched to a new branch 'feature-a'
x1lefg0r@x1lefg0r-OSX edutrack % git switch master
Switched to branch 'master'
Your branch and 'origin/master' have diverged,
and have 1 and 2 different commits each, respectively.
(use "git pull" if you want to integrate the remote branch with yours)
x1lefg0r@x1lefg0r-OSX edutrack % git checkout -b feature-b
Switched to a new branch 'feature-b'
x1lefg0r@x1lefg0r-OSX edutrack % git rebase --onto master 1ea8432 feature-a
Successfully rebased and updated refs/heads/feature-a.
x1lefg0r@x1lefg0r-OSX edutrack % git log --oneline feature-a
efccbc0 (HEAD -> feature-a, master, feature-b) chore: docs
cdf3046 chore: added readme lab
60c297f Revert "chore: added readme"
ecb84a2 chore: docs
c058d5c chore: added readme
6f111a4 feat: added openapi schema
af7f0fa refactor: added some ally
2d9b048 fix: fixed pagination page
1ea8432 fix: bugs
f4e2c0a fix: small bug with achievements
2e6724c feat: added ollama to seeder and fixes small bugs
ee6e0ba feat(frontend): added
360b491 chore: install shadcn
e5b282a chore: added shadcn
540d787 chore: init
b5ffb02 chore: init
x1lefg0r@x1lefg0r-OSX edutrack % git branch -d feature-a feature-b
error: cannot delete branch 'feature-a' used by worktree at '/Users/x1lefg0r/un/web/edutrack'
Deleted branch feature-b (was efccbc0).
x1lefg0r@x1lefg0r-OSX edutrack % git branch
* feature-a
  git-lab-demo
  master
x1lefg0r@x1lefg0r-OSX edutrack % git checkout git-lab-demo
Switched to branch 'git-lab-demo'
x1lefg0r@x1lefg0r-OSX edutrack % git branch -d feature-a feature-b
error: the branch 'feature-a' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D feature-a'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
error: branch 'feature-b' not found
x1lefg0r@x1lefg0r-OSX edutrack % git branch -D feature-a feature-b
error: branch 'feature-b' not found
Deleted branch feature-a (was efccbc0).
x1lefg0r@x1lefg0r-OSX edutrack %

```

Рисунок 17 – git log --oneline ветки до и после rebase --onto

18. Команда git blame

Для каждой строки файла показывает: хэш коммита, автора, дату и содержимое строки. Незаменимо для поиска того, кто и когда внёс конкретное изменение.

```

git blame README.md           # построчная история файла
git blame -L 1,10 README.md   # только строки 1–10
git blame --date=short README.md

```

```

x1lefg0r@x1lefg0r-OSX edutrack % git blame README.md
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 1) # EduTrack
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 2)
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 3) Веб-платформа для подготовки школьников к олимпиадам. Помогает находить олимпиады
и курсы по предметам, отслеживать прогресс и зарабатывать достижения за активность.
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 4)
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 5) ## Стек
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 6)
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 7) **Backend**
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 8) - Python 3.13, Django 4.2, Django REST Framework
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 9) - PostgreSQL (psycopg3)
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 10) - JWT-аутентификация - `djanoestframework-simplejwt`
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 11) - OpenAPI-документация - `drf-spectacular` → `/api/schema/swagger/`
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 12) - Профилирование запросов - `django-silk` → `/silk/`
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 13)
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 14) **Frontend**
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 15) - Next.js 16, React 19, TypeScript
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 16) - TanStack React Query - серверный стейт
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 17) - Tailwind CSS 4, shadcn/ui
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 18) - Архитектура - [Feature-Sliced Design](https://feature-sliced.design/)
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 19) - TypeScript-типы генерируются автоматически из OpenAPI схемы
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 20)
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 21) ## Быстрый старт
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 22)
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 23) ```bash
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 24) # 1. Запустить PostgreSQL
cdf3046c (x1lefg0r 2026-05-31 22:43:31 +0300) 25) docker-compose up -d
x1lefg0r@x1lefg0r-OSX edutrack %

```

Рисунок 18 – Вывод git blame README.md

19. Команда git bisect

Находит коммит, «сломавший» что-то, методом бинарного поиска. Указываются «плохой» и «хороший» коммиты — Git поочерёдно предлагает коммиты для проверки, сужая диапазон вдвое. Поддерживает автоматизацию через `git bisect run <команда>`.

```

git bisect start
git bisect bad                # текущий коммит — плохой
git bisect good 540d787      # этот коммит — хороший
# Git переключает HEAD на середину — проверяем и отвечаем:
git bisect good              # или: git bisect bad
# После нахождения первого плохого коммита:
git bisect reset              # выход из режима bisect

# Автоматизация:
git bisect start HEAD 540d787
git bisect run npm test
git bisect reset

```

```
● x1lefg0r@x1lefg0r-OSX edutrack % git bisect start HEAD 540d787
Already on 'git-lab-demo'
Bisecting: 6 revisions left to test after this (roughly 3 steps)
[2d9b0480d71bef6e3d07b8413e64b05918f28b86] fix: fixed pagination page
● x1lefg0r@x1lefg0r-OSX edutrack % git bisect run npm test
running 'npm' 'test'
npm error code ENOENT
npm error syscall open
npm error path /Users/x1lefg0r/un/web/edutrack/package.json
npm error errno -2
npm error enoent Could not read package.json: Error: ENOENT: no such file or directory, open '/Users/x1lefg0r/un/web/edutrack/package.json'
npm error enoent This is related to npm not being able to find a file.
npm error enoent
npm error A complete log of this run can be found in: /Users/x1lefg0r/.npm/_logs/2026-05-31T20_33_39_535Z-debug-0.log
error: bisect run failed: exit code 254 from 'npm' 'test' is < 0 or >= 128
● x1lefg0r@x1lefg0r-OSX edutrack % git bisect reset
Previous HEAD position was 2d9b048 fix: fixed pagination page
Switched to branch 'git-lab-demo'
● x1lefg0r@x1lefg0r-OSX edutrack % |
```

Рисунок 19 – Вывод git bisect с найденным «плохим» коммитом

Контрольные вопросы и ответы

В чём разница между git revert и git reset?

git revert создаёт новый коммит, отменяющий изменения, не трогая историю — безопасно для опубликованных веток. git reset перемещает HEAD назад, переписывая историю — подходит только для локальных, ещё не запущенных коммитов.

Что такое detached HEAD и когда он возникает?

Состояние, при котором HEAD указывает напрямую на хэш коммита, а не на ветку. Возникает при git checkout <hash>. Изменения в этом состоянии не привязаны ни к одной ветке и могут быть «потеряны» при переключении.

Зачем нужен git reflog?

git reflog хранит лог всех перемещений HEAD за последние 90 дней, включая те, что не видны в git log после reset --hard. Позволяет восстановить «потерянные» коммиты.

Чем git stash pop отличается от git stash apply?

Оба восстанавливают последнее спрятанное состояние, но pop удаляет его из стека, а apply — оставляет. apply полезен, когда нужно применить одно и то же состояние несколько раз.

Для чего используется git bisect?

Для поиска коммита, введшего регрессию, методом бинарного поиска. Позволяет быстро найти «виновный» коммит среди сотен, не проверяя каждый вручную. Поддерживает автоматизацию через git bisect run.